Apuntes Premium de Estudio

APIs RESTful, EF Core, Scaffolding y DTOs en .NET

Qué vas a encontrar: Documento reorganizado con enfoque pedagógico: bloques cortos, analogías, ejemplos mínimos y explicación del porqué y del cómo bajo el capó.

Perfil de lectura recomendado

- Lee una sección por bloque de 10 a 15 minutos.
- No memorices primero: entiende el modelo mental y luego vuelve al código.
- Si algo te cuesta, revisa primero el bloque 'Análisis profundo'; ahí está la lógica real que hace que todo encaje.

Mapa del documento

Módulo	Pregunta que responde
1	¿Qué significa pensar una API como REST y no como una colección de acciones?
2	¿Cómo se usan HTTP, status codes y respuestas profesionales en .NET?
3	¿Qué hace EF Core y por qué existe?
4	¿Cómo se modelan entidades y relaciones con DbContext?
5	¿Cómo conecto mi API a SQL Server con Docker o LocalDB?
6	¿Cuándo usar Code First, migraciones y Database First?
7	¿Qué me genera el scaffolding y qué debo revisar antes de confiar en él?
8	¿Cómo pruebo el CRUD real y por qué a veces se 'pierden' IDs?

Módulo	Pregunta que responde
9	¿Por qué Include trae datos, por qué aparecen null y por qué a veces revienta el JSON?
10	¿Para qué sirven los DTOs y la validación?
11	¿Cómo refactorizo controladores para que entren DTOs y salgan DTOs?
12	¿Con qué mapa mental me quedo al terminar la sección?

Regla pedagógica: Este documento prioriza claridad. Si una idea tiene dos niveles - concepto y mecánica interna - primero te la explico con una analogía y luego te la aterrizo con código.

1. Fundamentos REST: pensar en recursos, no en acciones

🎯 Objetivo

entender qué es REST, por qué no es un framework y cómo usarlo como modelo mental para diseñar APIs limpias y escalables.

🧠 El concepto

Imagina la recepción de un hotel. El recepcionista no 'adivina' lo que quieres: tú llegas, haces una petición concreta y entregas toda la información necesaria. REST funciona igual.

REST es un estilo de arquitectura. No es una librería ni un lenguaje. Su idea central es sencilla: exponer recursos mediante URLs y operar sobre esos recursos usando HTTP.

La clave está en pensar en sustantivos. En vez de diseñar rutas como /getHotels o /createHotel, diseñas recursos como /api/hotels y dejas que el verbo HTTP diga la intención.

Ejemplo mínimo viable: La misma ruta puede significar cosas distintas según el verbo HTTP que la acompaña.

HTTP

```
GET /api/hotels
POST /api/hotels
DELETE /api/hotels/3
```

🛠️ La parte técnica

Principios base de REST

Principio	Qué significa en lenguaje claro	Impacto práctico
Stateless	El servidor no guarda tu sesión entre peticiones	Cada request debe traer token y datos suficientes
Cliente-servidor	Frontend y backend están desacoplados	Puedes cambiar React, móvil o .NET sin rehacer todo
Basado en recursos	Diseñas URLs como recursos, no acciones	La API se vuelve predecible y fácil de consumir

🔍 Análisis profundo

Lo más importante bajo el capó es el principio stateless. Como el servidor no guarda memoria del cliente, cualquier request puede ser atendido por cualquier instancia del backend.

Eso vuelve la API escalable: si hoy atiendes 100 usuarios y mañana 10,000, puedes agregar más servidores detrás de un balanceador sin reescribir la lógica de negocio.

También vuelve el sistema más resistente. Si una instancia cae, otra puede contestar la siguiente petición porque toda la información útil viaja en el request.

🚦 Semáforo de aprendizaje

🟢 REST hace que tu API sea fácil de entender porque cada URL representa un recurso claro.

⚠️ si empiezas a meter verbos dentro de la ruta, la semántica se rompe y el frontend deja de poder adivinar el comportamiento.

🔴 confundir REST con 'usar JSON' es un error. Puedes devolver JSON y aun así tener una API mal diseñada.

✅ Cierre de progreso

Después de leer esto, ya eres capaz de explicar qué es REST, reconocer una URL bien diseñada y justificar por qué el modelo stateless favorece la escalabilidad.

2. HTTP en la práctica: verbos, status codes y respuestas profesionales

🎯 Objetivo

usar HTTP con intención, devolver códigos coherentes y escribir endpoints que comuniquen claramente qué pasó.

🧠 El concepto

Piensa en HTTP como el lenguaje de cortesía de la web. No solo importa lo que haces, también importa cómo lo comunicas.

Una API profesional no se limita a devolver datos; también devuelve contexto. Ese contexto vive en el verbo, en el status code y, a veces, en headers como Location.

Cuando el backend responde bien, el frontend puede mostrar mensajes correctos sin tener que adivinar.

> **Ejemplo mínimo viable:** Un POST bien hecho no responde con 200 por costumbre: responde con 201 Created e idealmente indica dónde quedó el nuevo recurso.

CSHARP

```csharp
[HttpPost]
public ActionResult<Hotel> Post([FromBody] Hotel newHotel)
{
    if (newHotel == null)
        return BadRequest("Datos inválidos.");

    if (hotels.Any(h => h.Id == newHotel.Id))
        return Conflict("El ID del hotel ya existe.");

    hotels.Add(newHotel);
    return CreatedAtAction(nameof(Get), new { id = newHotel.Id }, newHotel);
}
```

🛠️ La parte técnica

Verbos HTTP más usados

Verbo	Acción	Uso típico
GET	Obtener	Consultar lista o detalle

Verbo	Acción	Uso típico
POST	Crear	Crear un recurso nuevo
PUT	Actualizar completo	Reemplazar o actualizar recurso existente
DELETE	Eliminar	Borrar un recurso

TABLA COMPARATIVA

409 Conflict -> la petición es válida, pero choca con un dato existente.
400 Bad Request -> la petición viene mal formada o incompleta.

🔍 Análisis profundo

ActionResult<T> es preferible a devolver la entidad directamente porque te da control total del contrato HTTP. Puedes responder 200, 201, 400, 404 o 409 sin forzar hacks extraños.

CreatedAtAction hace dos cosas valiosas: devuelve 201 Created y agrega el header Location con la URL del recurso recién creado. Eso ayuda a clientes, pruebas y documentación Swagger.

nameof(Get) evita strings mágicos. Si renombraras el método Get en el futuro, el compilador te avisaría en vez de dejar un texto roto perdido dentro del código.

Opción	Ventaja	Riesgo si no la usas
ActionResult<T>	Controla semántica HTTP y errores	La API responde siempre 'bonito', pero poco expresiva
CreatedAtAction	Declara creación real y ruta del recurso	Pierdes semántica REST y trazabilidad
409 Conflict	Distingue conflicto lógico de error técnico	El frontend no sabe si debe corregir formato o contenido

🚦 Semáforo de aprendizaje

🟢 usar verbos y códigos correctos hace que Swagger, Postman y el frontend trabajen mejor contigo.

⚠️ muchos equipos devuelven 200 para todo porque 'funciona'; eso simplifica hoy pero confunde mañana.

🔴 devolver 400 cuando el problema real es un duplicado borra una pista importantísima para el cliente.

✅ Cierre de progreso

Después de leer esto, ya eres capaz de elegir el verbo correcto, diferenciar 400 de 409 y escribir un POST que responda de forma RESTful.

3. Entity Framework Core: el traductor entre C# y SQL

🎯 Objetivo

entender por qué existe EF Core y cuál es su papel real entre tus clases de C# y la base de datos relacional.

🧠 El concepto

Imagina un traductor simultáneo. Tú hablas en objetos C#, pero la base de datos entiende SQL. EF Core se sienta en medio y traduce ambos mundos.

Gracias a ese traductor, tú puedes trabajar con clases, propiedades y LINQ, mientras EF Core genera consultas, rastrea cambios y persiste datos.

Eso no significa magia absoluta. EF Core automatiza mucho, pero sigue obedeciendo reglas muy concretas: entidades, contexto, proveedor y migraciones.

> **Ejemplo mínimo viable:** Escribes una consulta en LINQ y EF Core la convierte a SQL por ti.

CSHARP

```
var hotels = await context.Hotels
    .Where(h => h.Rating >= 4)
    .ToListAsync();
```

🛠️ La parte técnica

Piezas básicas de EF Core

Componente	Qué hace	Cómo debes imaginarlo
Entity	Representa una tabla	La clase que modela tus datos
DbContext	Gestiona conexión y seguimiento	El cerebro que coordina todo
LINQ	Expresa consultas en C#	Tu lenguaje de preguntas
Provider	Habla con SQL Server, PostgreSQL, etc.	El acento del traductor
Migration	Versiona cambios de esquema	El historial de evolución de la base

🔍 Análisis profundo

Cuando escribes LINQ, EF Core no siempre ejecuta la consulta de inmediato. Primero construye una representación interna del query. La ejecución real ocurre cuando materializas resultados con ToListAsync, FirstOrDefaultAsync y métodos similares.

Además, EF Core hace change tracking: recuerda qué entidades cargó y qué propiedades cambiaron. Por eso SaveChanges puede generar UPDATE o INSERT sin que tú escribas SQL a mano.

El enfoque Code First parte del código y construye la base. Database First hace la ingeniería inversa: parte de una base existente y genera clases. Ambos usan al mismo traductor, pero empiezan desde lados distintos.

🚦 Semáforo de aprendizaje

🟢 EF Core acelera desarrollo y te deja pensar en dominio antes que en SQL repetitivo.

⚠️ seguir usando EF Core no te exime de pensar en performance; un LINQ mal escrito puede traducirse a un SQL caro.

🔴 creer que EF Core 'reemplaza' el conocimiento de SQL es un error. Lo que hace es ocultarte parte del cableado, no la física del sistema.

✅ Cierre de progreso

Después de leer esto, ya eres capaz de describir el papel de Entity, DbContext, Provider y Migration, y entender cómo LINQ termina convertido en SQL.

4. Modelado y DbContext: cómo tu código define la base de datos

🎯 Objetivo

entender cómo EF Core detecta claves, relaciones y tablas a partir de tus clases, y por qué DbContext es el corazón operativo del ORM.

🧠 El concepto

Piensa en tus entidades como planos y en el DbContext como el arquitecto que los interpreta. Las entidades describen qué existe; el contexto decide cómo conversar con la base y cómo ensamblar todo.

Cuando sigues convenciones sencillas - Id, CountryId, propiedades de navegación - EF Core puede deducir muchísimo sin configuración manual.

Eso vuelve el código más limpio y fácil de mantener, siempre que respetes las reglas básicas del modelado.

Ejemplo mínimo viable: Una relación uno a muchos puede definirse casi por convención.

CSHARP

```
public class Country
{
    public int CountryId { get; set; }
    public string Name { get; set; }
    public List<Hotel> Hotels { get; set; } = new();
}

public class Hotel
{
    public int Id { get; set; }
    public string Name { get; set; }
    public int CountryId { get; set; }
    public Country Country { get; set; }
}
```

🛠️ La parte técnica

Convenciones que EF Core reconoce solo

Convención	Qué detecta EF Core	Ejemplo
Id o [Entidad]Id	Clave primaria	Id, CountryId, HotelId
Propiedad int + navegación	Clave foránea	CountryId + Country
List<T> en el lado padre	Relación uno a muchos	List<Hotel> Hotels
DbSet<T>	Tabla incluida en el modelo	DbSet<Country> Countries

CSHARP

```csharp
public class HotelListingDbContext : DbContext
{
    public HotelListingDbContext(DbContextOptions<HotelListingDbContext> options)
        : base(options)
    {
    }

    public DbSet<Country> Countries { get; set; }
    public DbSet<Hotel> Hotels { get; set; }
}
```

🔍 Análisis profundo

DbContext representa una sesión activa con la base. Gestiona conexión, seguimiento de cambios, ejecución de consultas y persistencia. Sin él, EF Core no tiene 'lugar' donde operar.

Cuando declaras un DbSet, no estás creando datos todavía, pero sí le estás diciendo al modelo: 'esta entidad forma parte de mi universo'. Eso tiene consecuencias directas al generar migraciones.

La relación Hotel -> Country existe porque el lado hijo guarda la foreign key y el lado padre expone la colección. EF Core une ambos extremos y deduce cardinalidad.

🚦 Semáforo de aprendizaje

🟢 si sigues convenciones simples, el modelado inicial es muy rápido y limpio.

⚠️ si nombras propiedades de formas poco convencionales, probablemente necesitarás atributos o Fluent API.

🔴 olvidar inicializar listas de navegación es una receta clásica para NullReferenceException.

✅ Cierre de progreso

Después de leer esto, ya eres capaz de modelar entidades básicas, leer una relación uno a muchos y explicar por qué DbContext es el centro de operaciones de EF Core.

5. Conexión a SQL Server: Docker, LocalDB y cadena de conexión

🎯 Objetivo

entender cómo se conecta una API .NET a SQL Server y cuál es el papel de Docker, LocalDB y la connection string.

🧠 El concepto

Imagina que tu base de datos es una casa cerrada. La connection string es la dirección completa más la llave correcta de entrada.

Si no sabes dónde está el servidor, qué base usar, qué usuario entra y con qué contraseña, EF Core no puede hacer nada útil.

En desarrollo tienes dos caminos muy comunes: ejecutar SQL Server en Docker o usar LocalDB si trabajas en Windows con Visual Studio.

Ejemplo mínimo viable: Registrar el contexto en Program.cs usando una cadena de conexión definida en appsettings.json.

CSHARP

```csharp
var connectionString = builder.Configuration
    .GetConnectionString("HotelListingDbConnectionString");

builder.Services.AddDbContext<HotelListingDbContext>(options =>
{
    options.UseSqlServer(connectionString);
});
```

🛠️ La parte técnica

Comparación rápida de entornos de desarrollo

Escenario	Ventaja	Dato clave de conexión
Docker	Consistencia multiplataforma	Server=localhost,1400; User Id=sa; Password=...
LocalDB	Muy cómodo en Windows + Visual Studio	Server=(localdb)\\MSSQLLocalDB; Trusted_Connection=True

BASH

```bash
docker run -e "ACCEPT_EULA=Y" -e "MSSQL_SA_PASSWORD=TuPasswordFuerte123!" \
  -p 1400:1433 --name sql_server_container \
  -d mcr.microsoft.com/mssql/server
```

🔍 Análisis profundo

Docker encapsula SQL Server dentro de un contenedor. No estás 'instalando' SQL Server en tu máquina como antes; estás levantando un entorno aislado, reproducible y desechable.

La connection string junta varios datos que EF Core necesita pasarle al provider: host, puerto, base, tipo de autenticación y parámetros de conexión.

El paso AddDbContext registra el contexto en el contenedor de dependencias. A partir de ahí, ASP.NET Core puede inyectarlo en controladores y servicios automáticamente.

🚦 Semáforo de aprendizaje

🟢 Docker te evita el clásico 'en mi máquina sí funciona' y vuelve más estable el entorno de desarrollo.

⚠️ en JSON debes escapar la barra invertida de LocalDB con doble slash; en la terminal no.

🔴 subir contraseñas reales a un repositorio público es un error grave, aunque solo sea 'para salir del paso'.

✅ Cierre de progreso

Después de leer esto, ya eres capaz de leer una cadena de conexión, distinguir Docker de LocalDB y registrar correctamente el DbContext en Program.cs.

6. Migraciones y Database First: dos rutas para llegar a la misma base

🎯 Objetivo

entender cómo evolucionar la base con migraciones y cuándo conviene usar Database First con scaffolding.

🧠 El concepto

Piensa en dos formas de construir una casa. En Code First dibujas el plano y luego levantas la casa. En Database First entras a una casa ya construida y haces un levantamiento arquitectónico para convertirla en planos.

Las migraciones son el historial de cambios del esquema. No son simples archivos decorativos: son instrucciones concretas para llevar la base desde una versión a otra.

Scaffolding, en cambio, hace el camino inverso. Lee la base y genera entidades y contexto automáticamente.

Ejemplo mínimo viable: Flujo típico con Code First.

BASH

```bash
dotnet ef migrations add Initial

dotnet ef database update
```

🛠️ La parte técnica

Cuándo usar cada enfoque

Enfoque	Punto de partida	Ideal para
Code First	Clases C#	Proyectos nuevos o greenfield
Database First	Base existente	Sistemas legacy o integración con BD ya definida

BASH

```bash
dotnet ef dbcontext scaffold "<CONNECTION_STRING>" Microsoft.EntityFrameworkCore.SqlServer \
  --context-dir Data \
  --output-dir Models
```

🔍 Análisis profundo

Una migración guarda el antes y el después del modelo. EF Core compara el modelo actual con el snapshot previo y genera operaciones como CreateTable, AddColumn o AddForeignKey.

Scaffolding no es lo mismo que migración. No versiona el esquema, solo genera código a partir del estado actual de la base.

Si vuelves a hacer scaffolding con --force, sobrescribes entidades y contexto. Eso es útil para sincronizar cambios, pero peligroso si habías modificado esos archivos a mano.

🚦 Semáforo de aprendizaje

🟢 las migraciones pequeñas y frecuentes son fáciles de entender, revisar y revertir.

⚠️ Database First es muy útil, pero te obliga a ser cuidadoso al regenerar modelos si agregaste lógica manual.

🔴 mezclar Code First y Database First sin estrategia suele terminar en desorden y archivos inconsistentes.

✅ Cierre de progreso

Después de leer esto, ya eres capaz de explicar la diferencia entre migración y scaffolding, y elegir conscientemente entre Code First y Database First.

7. Scaffolding de controladores: cuándo te acelera y qué te genera

🎯 Objetivo

entender qué hace el scaffolding de controladores en ASP.NET Core, por qué es útil para arrancar rápido y qué partes debes revisar antes de llevarlo a producción.

🧠 El concepto

El scaffolding se parece a una casa prefabricada. Te entrega una estructura habitable muy rápido, pero no necesariamente la distribución final que querrías para una casa buena y duradera.

En proyectos internos, demos o pruebas rápidas, ese esqueleto puede ahorrarte mucho tiempo. En producción casi siempre necesitas refactorizar.

Lo importante no es solo 'que genere código', sino entender qué decidió generar y por qué.

> **Ejemplo mínimo viable:** Opción de Visual Studio que genera un controlador conectado a EF Core.

TEXTO

```
API Controller with actions, using Entity Framework
```

🛠️ La parte técnica

Opciones habituales de scaffolding en Visual Studio

Opción	Qué genera	Cuándo conviene
API Controller - Empty	Solo la clase vacía	Cuando quieres escribir todo a mano
API Controller with read/write actions	Métodos de ejemplo con datos falsos	Prototipos sin base de datos
API Controller with actions, using Entity Framework	CRUD conectado a DbContext	Cuando ya tienes EF Core configurado

CSHARP

```csharp
public class CountriesController : ControllerBase
{
    private readonly HotelListingDbContext _context;

    public CountriesController(HotelListingDbContext context)
    {
        _context = context;
    }
}
```

🔍 Análisis profundo

ASP.NET Core usa inyección de dependencias nativa. El controlador no crea su propio contexto; el contenedor lo resuelve y se lo inyecta en el constructor.

El binding de rutas también ocurre automáticamente. Cuando tienes [HttpGet("{id}")], el framework intenta conectar el segmento de URL con el parámetro del método.

En el PUT scaffolded suele aparecer _context.Entry(entity).State = EntityState.Modified. Eso le dice a EF Core: 'marca esta entidad completa como modificada', aunque no sepas exactamente qué propiedad cambió.

🚦 Semáforo de aprendizaje

🟢 el scaffolding te da un CRUD funcional muy rápido y te deja enfocarte primero en la tubería completa.

⚠️ si tienes relaciones padre-hijo, conviene scaffoldear primero las entidades padre para no bloquearte al probar.

🔴 llevar el controlador generado tal cual a producción sin DTOs, validación y limpieza es pedir problemas.

✅ Cierre de progreso

Después de leer esto, ya eres capaz de identificar qué hace el scaffolding, cuándo usarlo y qué revisar en el controlador generado antes de confiar en él.

8. Probar el CRUD real: errores útiles, IDs saltados y relaciones

🎯 Objetivo

verificar que el controlador realmente habla con la base de datos y entender errores típicos como claves foráneas inválidas e identity gaps.

🧠 El concepto

Probar el CRUD es como abrir la llave del agua antes de cerrar la pared. No basta con que el código compile; necesitas ver que la tubería llega hasta SQL Server y vuelve con la respuesta correcta.

Cuando haces pruebas reales, aparecen detalles que no se ven en una demo: errores 400, 404, conflictos de clave foránea y el famoso misterio de los IDs que se saltan números.

Lejos de ser molestias, esos errores te enseñan cómo protege la base de datos la integridad del sistema.

> **Ejemplo mínimo viable:** Crear un hotel con un countryId válido y luego provocar un error usando un countryId inexistente.

JSON

```
POST /api/hotels
{
  "name": "Sandals Resort",
  "address": "Negril",
  "rating": 4.5,
  "countryId": 1
}

POST /api/hotels
{
  "name": "Ghost Hotel",
  "countryId": 99
}
```

🛠️ La parte técnica

Qué observar en cada operación

- GET correcto: suele devolver 200 OK y lista vacía si la tabla todavía no tiene registros.
- POST correcto: debe devolver 201 Created y el ID generado por la base.
- PUT correcto: normalmente devuelve 204 No Content si actualiza sin devolver cuerpo.
- DELETE correcto: también suele devolver 204 cuando la eliminación fue exitosa.

Concepto clave: Si un INSERT falla por integridad referencial, SQL Server puede haber reservado el siguiente ID antes de descubrir el error. Por eso aparecen huecos numéricos.

🔍 Análisis profundo

El identity gap ocurre porque el contador de identidad no 'rebobina' cuando una operación falla. SQL Server prioriza concurrencia y rendimiento, no numeración perfecta sin huecos.

Eso significa que los IDs deben verse como identificadores únicos, no como un contador exacto de filas insertadas con éxito.

Además, EF Core parametriza consultas e inserts. En lugar de concatenar strings peligrosos, usa parámetros como @p0, lo que reduce el riesgo de SQL Injection.

Situación	Lo que parece	Lo que realmente pasó
El siguiente hotel salta de 2 a 5	'Se perdió información'	Hubo intentos fallidos que consumieron valores Identity
POST con countryId inexistente	'La API falló porque sí'	La base bloqueó un hijo huérfano por foreign key
GET devuelve []	'No funciona nada'	La conexión sí funciona; solo no hay datos todavía

🚦 Semáforo de aprendizaje

🟢 probar con requests reales te confirma que el flujo completo - API, EF Core y SQL - está vivo.

⚠️ los IDs saltados son normales; pueden asustar al principio si los interpretas como contador continuo.

🔴 intentar 'rellenar huecos' de IDs manualmente es una mala práctica y suele traer más problemas que soluciones.

✅ Cierre de progreso

Después de leer esto, ya eres capaz de probar GET, POST, PUT y DELETE con criterio, entender por qué fallan las claves foráneas y explicar por qué aparecen IDs saltados.

9. Datos relacionados y serialización: Include, null y bucles infinitos

🎯 Objetivo

entender cómo cargar datos relacionados con Include, por qué a veces aparecen propiedades null y por qué una relación bidireccional puede romper la serialización JSON.

🧠 El concepto

Un Country aislado es como un solista: se entiende por sí solo. Un Hotel es como una banda: depende de otras piezas, en este caso de Country.

Por defecto EF Core es conservador. Si pides Hoteles, no traerá automáticamente el Country entero a menos que se lo indiques.

Eso explica por qué a veces ves countryId con valor correcto, pero la propiedad Country aparece en null.

> **Ejemplo mínimo viable:** Cargar un hotel e incluir el país asociado.

CSHARP

```
var hotel = await _context.Hotels
    .Include(h => h.Country)
    .FirstOrDefaultAsync(h => h.Id == id);
```

🛠️ La parte técnica

Soluciones frecuentes al problema

Problema	Qué lo causa	Solución típica
Country viene null	No cargaste datos relacionados	Usar Include o proyectar con Select
Error de ciclo JSON	Hotel apunta a Country y Country vuelve a Hotels	Configurar IgnoreCycles o usar DTOs
Consulta pesada en listados	Incluyes demasiado para una lista	Reservar detalles completos para GetById

CSHARP

```csharp
builder.Services.AddControllers().AddJsonOptions(options =>
{
    options.JsonSerializerOptions.ReferenceHandler =
        System.Text.Json.Serialization.ReferenceHandler.IgnoreCycles;
});
```

🔍 Análisis profundo

Include se traduce en un JOIN SQL. No es magia aparte: simplemente le estás pidiendo a EF Core que te traiga en la misma consulta la entidad relacionada.

El bucle infinito aparece cuando serializas un grafo de objetos bidireccional. Hotel contiene Country, Country contiene Hotels, y esos hoteles vuelven a contener Country. El serializador entra en un espejo frente a otro espejo.

IgnoreCycles corta ese abrazo mortal, pero no es la solución arquitectónicamente más limpia. La solución profesional suele ser usar DTOs y decidir exactamente qué datos salen.

🚦 Semáforo de aprendizaje

🟢 Include resuelve rápido el problema de datos relacionados y hace legible el código.

⚠️ poner Include en todos los listados puede empeorar rendimiento y traer datos que no necesitas.

🔴 confiar solo en IgnoreCycles para todo el diseño de salida deja un JSON funcional, pero poco elegante y difícil de controlar.

✅ Cierre de progreso

Después de leer esto, ya eres capaz de usar Include cuando corresponde, explicar por qué aparece un null en navegación y entender la causa real del error de referencia cíclica.

10. DTOs y validación: el cortafuegos de tu API

🎯 Objetivo

entender por qué no conviene exponer entidades directamente y cómo DTOs y validación protegen tu API, tu base y tu contrato con el cliente.

🧠 El concepto

Imagina la cocina de un restaurante. Las entidades son la cocina real: tienen ingredientes, cuchillos, reglas internas y detalles que el cliente nunca debería ver. El DTO es el mesero: toma solo lo necesario y lo presenta bien.

La validación funciona como el control de seguridad del aeropuerto. Si el dato está mal, se rechaza antes de que llegue a la pista donde están los recursos caros: el controlador, EF Core y la base de datos.

Juntas, estas dos capas hacen algo muy poderoso: filtran lo que entra y dan forma a lo que sale.

Ejemplo mínimo viable: DTO de creación con reglas básicas de validación.

CSHARP

```csharp
public class CreateHotelDto
{
    [Required]
    public required string Name { get; set; }

    [Required]
    public required string Address { get; set; }

    [Required]
    [Range(1, 5)]
    public double Rating { get; set; }

    [Required]
    [Range(1, int.MaxValue)]
    public int CountryId { get; set; }
}
```

🛠️ La parte técnica

Tipos de DTO más comunes

DTO	Para qué sirve	Qué suele contener
CreateDto	Entrada para crear	Campos obligatorios, sin Id
UpdateDto	Entrada para actualizar	Id + campos editables
GetDto	Salida de lectura	Campos visibles al cliente
DetailDto	Salida enriquecida	Datos propios + datos relacionados

🔍 Análisis profundo

El problema del overposting ocurre cuando dejas que el cliente envíe la entidad completa. Si la entidad tiene campos sensibles o que el cliente no debería tocar, esos campos podrían ser llenados maliciosamente.

La validación con Data Annotations corre antes de entrar al controlador. Si falla, ASP.NET Core devuelve 400 Bad Request y rellena ProblemDetails con mensajes por campo.

Eso ahorra trabajo a la base. Una petición inválida ni siquiera llega a intentar un INSERT, lo que mejora experiencia del usuario y reduce operaciones inútiles.

Mecanismo	Cuándo actúa	Para qué protege
[Required], [Range], [MaxLength]	Runtime, al recibir la petición	La entrada HTTP
required (keyword)	Compile time	La creación de objetos dentro de C#
DTO	Diseño del contrato	Lo que entra y sale de la API

🚦 Semáforo de aprendizaje

🟢 DTOs te dan seguridad, claridad y control sobre el payload.

⚠️ si mezclas reglas de validación de la API dentro de las entidades, acoplas demasiado la base con la capa de transporte.

🔴 exponer entidades directamente en endpoints públicos te deja vulnerable a sobrepublicación y a contratos difíciles de mantener.

✅ Cierre de progreso

Después de leer esto, ya eres capaz de justificar el uso de DTOs, ubicar la validación en la capa correcta y distinguir entre required del compilador y Required del runtime.

11. Refactorizar controladores con DTOs: proyección, mapping y relaciones anidadas

🎯 Objetivo

mover la API de un prototipo funcional a una forma más profesional: entran DTOs, salen DTOs y la consulta trae solo lo necesario.

🧠 El concepto

Aquí puedes pensar en dos herramientas mentales. Include es como una fotocopiadora: tiende a traerse el objeto completo relacionado. Select es como una tijera: recorta solo las partes que realmente quieres enseñar.

Cuando refactorizas a DTOs, el objetivo no es únicamente 'hacer más archivos', sino ganar control. Qué entra, qué sale y qué nombre tienen las cosas deja de depender de la entidad interna.

También aparece un trabajo repetitivo: mapear DTO a Entity y viceversa. Al principio se hace a mano. Más adelante herramientas como AutoMapper ayudan a automatizarlo.

Ejemplo mínimo viable: Proyectar un HotelDetailsDto sin devolver la entidad completa.

CSHARP

```csharp
var hotel = await _context.Hotels
    .Where(h => h.Id == id)
    .Select(h => new HotelDetailsDto(
        h.Id,
        h.Name,
        h.Address,
        h.Rating,
        h.CountryId,
        h.Country.Name))
    .FirstOrDefaultAsync();
```

🛠️ La parte técnica

Mapping manual típico

CSHARP

```
var hotel = new Hotel
{
    Name = dto.Name,
    Address = dto.Address,
    Rating = dto.Rating,
    CountryId = dto.CountryId
};
```

Regla de oro al proyectar

> **Regla:** Primero filtra con Where sobre propiedades reales de la entidad y después proyecta con Select al DTO. Filtrar 'después' del DTO suele complicar la traducción a SQL.

🔍 Análisis profundo

Cuando en un Select tocas h.Country.Name, EF Core entiende que necesita un JOIN aunque tú no hayas escrito Include. Eso es una gran ventaja de la proyección.

Si usas Include y luego Select, normalmente el Select manda: estás diciendo exactamente qué columnas quieres materializar, así que traer el objeto completo se vuelve redundante.

En relaciones anidadas, como Country con una lista de Hoteles, puedes proyectar dentro de otra proyección. Es un Select dentro de otro Select. Eso permite componer DTOs limpios sin exponer entidades.

Estrategia	Ventaja	Costo o riesgo
Include + entidad	Rápido para arrancar	Payload grande y ciclos más probables
Select + DTO	Payload limpio y preciso	Más trabajo de proyección o mapping
Mapping manual	Control total y explícito	Código repetitivo y propenso a omisiones

🚦 Semáforo de aprendizaje

🟢 la proyección con DTOs produce respuestas claras, planas y fáciles de consumir.

⚠️ el mapping manual es didáctico, pero a medida que crecen las propiedades se vuelve tedioso.

🔴 devolver entidades mezcladas con DTOs dentro del mismo controlador rompe consistencia y complica mantenimiento.

✅ Cierre de progreso

Después de leer esto, ya eres capaz de proyectar a DTOs, mapear entradas de forma segura y explicar por qué Select te da más control que Include cuando diseñas salidas.

12. Mapa mental final y checklist de buenas prácticas

🎯 Objetivo

consolidar las ideas principales para que puedas estudiar con un mapa claro y recordar qué revisar cada vez que construyas una API nueva.

🧠 El concepto

La mejor forma de cerrar este tema no es repitiendo definiciones, sino conectando piezas. REST define el contrato externo. HTTP da semántica. EF Core conecta objetos con SQL. DTOs y validación controlan qué entra y qué sale.

Si te pierdes, vuelve siempre a la pregunta correcta: ¿estoy diseñando un recurso, hablando con claridad por HTTP, modelando bien la relación con la base y protegiendo mi contrato con DTOs?

Con ese mapa, cada decisión técnica deja de sentirse arbitraria.

Checklist rápido para una API nueva: Usa esta secuencia cada vez que levantes un endpoint nuevo.

TEXTO

```
1) Diseña el recurso y la ruta.
2) Elige el verbo HTTP correcto.
3) Define qué DTO entra y qué DTO sale.
4) Valida antes de tocar la base.
5) Usa EF Core con una consulta clara.
6) Devuelve el status code correcto.
7) Prueba el flujo real con datos válidos e inválidos.
```

🛠️ La parte técnica

Checklist de revisión

Capa	Pregunta de control	Señal de que vas bien
REST	¿La URL representa un recurso?	Usas sustantivos y no verbos en la ruta
HTTP	¿El código de respuesta dice la verdad?	201 para crear, 404 para no encontrado, 409 para conflicto

Capa	Pregunta de control	Señal de que vas bien
EF Core	¿La consulta trae solo lo necesario?	Usas Include o Select con intención
DTOs	¿El cliente solo ve lo que debe ver?	Entran y salen DTOs, no entidades desnudas
Validación	¿Lo inválido muere antes de la base?	400 claros con ProblemDetails

🔍 Análisis profundo

Tu avance real no está en recordar todos los métodos, sino en reconocer el patrón. Casi todos los problemas de esta sección se reducen a cuatro tensiones: claridad del contrato, control del payload, costo de la consulta y seguridad de la entrada.

Si mantienes esas cuatro tensiones a la vista, podrás decidir mejor cuándo usar scaffolding, cuándo usar Include, cuándo proyectar, cuándo crear un DTO adicional y cuándo conviene refactorizar.

La sensación de progreso viene de eso: ya no miras piezas aisladas. Empiezas a ver el sistema completo.

🚦 Semáforo de aprendizaje

🟢 con un mapa mental claro, estudiar .NET deja de sentirse como una lista infinita de comandos.

⚠️ si quieres avanzar más rápido, evita quedarte solo en recetas; vuelve siempre al porqué de cada capa.

🔴 copiar scaffolding, pegar connection strings o usar DTOs por moda sin entender el motivo genera conocimiento frágil.

✅ Cierre de progreso

Después de leer esto, ya eres capaz de relacionar REST, HTTP, EF Core, validación y DTOs dentro de una sola arquitectura mental y estudiar futuras APIs con más criterio.

Siguiente paso sugerido

Si conviertes estos apuntes a un flujo de práctica, la mejor secuencia es:

- Primero crea Countries y Hotels con Code First.
- Después prueba CRUD simple con scaffolding.
- Luego rompe a propósito una relación para ver el error de clave foránea.
- Acto seguido aplica DTOs y validación.
- Por último refactoriza un GET con proyección y compara la diferencia entre Include y Select.

Idea final: Estudiar con intención significa esto: alternar una capa conceptual con una capa práctica. Si solo lees, olvidas. Si solo copias código, no entiendes.